

SOFTWARE ENGINEERING

منهجيات البرمجيات

Abdulfattah Alfarra

OUTLINE

- المنهجية □ أنشطة دورة حياة البرمجيات □ نماذج عملية البرمجيات
- العامة • البناء والإصلاح • الشلال • التدريجي • القائم على المكونات □
- Agile • البرمجة المتطرفة

Methodology

المنهجية هي عملية رسمية أو مجموعة من **الممارسات** لإنشاء البرامج.

□ **المنهجية** سلسلة من الأساليب أو التقنيات ذات الصلة.

Elements of Methodology



Elements of Methodology

□ **الأدوار:** من توظف، ماذا تفعل
توظيفهم، ما هي المهارات التي يحتاجونها
من المفترض أن يكون لديك.
□ مثل مدير المشروع، المصمم،
مطور، مختبر إلخ ،

Elements of Methodology

المهارات : المهارات اللازمة للأدوار. الكفاءة الشخصية للشخص الذي يشغل منصبًا ما هي نتاج تدريبه وموهبته. يحضر المبرمجون دورات لتعلم مهارات البرمجة الكائنية، و برمجة جافا، واختبار الوحدات. يتعلم مصممو واجهات المستخدم كيفية إجراء اختبارات قابلية الاستخدام وإنشاء النماذج الأولية الورقية . يتعلم المديرون مهارات إجراء المقابلات، والتحفيز، والتوظيف، وإدارة مهام المسار الحرج.

Elements of Methodology

السمات **الشخصية** المتوقعة من الموظف. يجب أن يكون مدير المشروع بارعًا في التعامل مع الناس، وأن يتمتع مصمم واجهات المستخدم بمواهب بصرية طبيعية وتعاطف مع سلوك المستخدم، وأن يتمتع مصمم البرامج الموجهة للكائنات بقدرات تجريد جيدة، وأن يجيد المستشار شرح الأمور.

Elements of Methodology

□ الفرق: الأدوار التي تعمل معًا في ظل ظروف مختلفة.

□ قد يكون هناك فريق واحد فقط في مشروع صغير.

في أي مشروع كبير، من المرجح أن يكون هناك فرق متعددة ومتداخلة، بعضها يهدف إلى ربط تقنيات محددة وبعضها يهدف إلى توجيه المشروع أو بنية النظام.

Elements of Methodology

التقنيات : الإجراءات المحددة التي يستخدمها الأشخاص لإنجاز المهام. بعضها ينطبق على شخص واحد (كتابة حالة استخدام، الإدارة بالتجول، تصميم فئة أو حالة اختبار)، بينما يستهدف بعضها الآخر مجموعات من الأشخاص (عرض مشروع، جلسات تخطيط جماعية).

Elements of Methodology

□ **الأنشطة:** كيف يقضي الأشخاص أيامهم.
التخطيط، والبرمجة، والاختبار، والاجتماع هي أنشطة نموذجية.

Elements of Methodology

العملية : كيفية ترابط الأنشطة مع بعضها البعض بمرور الوقت، مع مراعاة الشروط المسبقة واللاحقة للأنشطة (على سبيل المثال، تُجرى مراجعة للتصميم بعد يومين من إرسال المواد إلى المشاركين، وتُنتج قائمة بتوصيات للتحسين). تُركز المنهجيات المُركّزة على العملية على سير العمل بين أعضاء الفريق.

Elements of Methodology

منتجات **العمل**: ما يُنشئه شخص ما. قد يكون منتج العمل قابلاً للاستخدام مرة واحدة، كما هو الحال مع بطاقات تصميم CRC، أو قد يكون دائماً نسبياً، مثل دليل الاستخدام أو الكود المصدري.

Elements of Methodology

المعالم : أحداث تُشير إلى التقدم أو الإنجاز. بعض المعالم هي مجرد تأكيدات على إنجاز مهمة، وبعضها الآخر يتضمن نشر مستندات أو أكواد.

يتميز الحدث المهم بخاصيتين رئيسيتين: حدوثه في لحظة زمنية، وإما تحقيقه بالكامل أو عدمه (لا جزئيًا).
نشر الوثيقة أو عدم نشرها، تسليم الكود أو عدم تسليمه، عقد الاجتماع أو عدم عقده.

Elements of Methodology

□ **المعايير:** الاتفاقيات التي يتبناها الفريق لأدوات معينة، ومنتجات العمل، وسياسات القرار.

قد يُعلن معيار الترميز هذا: "لكل دالة تعليق رأسي..." وقد يكون معيار اللغة هذا: "سنستخدم جافا محمولة بالكامل". قد يكون معيار الرسم لمخططات الفئات هذا: "عرض الأساليب العامة للوظائف الدائمة فقط . "

□ قد يكون معيار الأداة على هذا النحو: "سنستخدم Microsoft مشروع، معًا/ج، وحدة "...،

Elements of Methodology

□ **الجودة:** قد تشير الجودة إلى الأنشطة أو منتجات العمل.

Elements of Methodology

قيم الفريق : تخضع بقية عناصر المنهجية لمنظومة قيم الفريق. فالفريق الطموح الذي يعمل على قيم تسويقية سريعة سيعمل بشكل مختلف تمامًا عن الفريق الذي يُقدّر العائلات ويعود إلى المنزل في وقت منتظم كل ليلة.

Process Life Cycle

□ تمر المنتجات البرمجية بمراحل عديدة أثناء نضجها من المفهوم الأولي إلى المنتج النهائي

□ تسمى سلسلة المراحل بنموذج العملية أو دورة الحياة . □ العملية عبارة عن سلسلة من الخطوات التي يتم تنفيذها من أجل

غرض معين

Process Life Cycle

تعريف:

نموذج البرمجيات أيضًا (نموذج دورة الحياة ونموذج العملية) هو وصف لتسلسل الأنشطة التي يتم تنفيذها في مشروع هندسة البرمجيات، والترتيب النسبي لهذه الأنشطة.

Process Life Cycle

□ يوفر إطارًا عامًا ثابتًا يمكن تخصيصه لمشروع محدد

□ يتم اختيار نموذج العملية للهندسة البرمجية بناءً على طبيعة المشروع والتطبيق والأساليب والأدوات التي سيتم استخدامها والضوابط والمنتجات المطلوبة

Software Life Cycle Activities

الأنشطة الرئيسية في مشروع البرمجيات هي: **تحديد المتطلبات** يعمل محلل النظام مع مستخدمي البرمجيات لتوضيح متطلبات النظام التفصيلية. تتضمن الأسئلة تنسيق بيانات الإدخال، والشكل المطلوب لأي شاشات إخراج، والتحقق من صحة البيانات

Software Life Cycle Activities

□ التحليل □ تأكد من فهمك الكامل للمشكلة قبل البدء في تصميم أو برمجة حل

□ تقييم الأساليب المختلفة للتصميم

Software Life Cycle Activities

□ **التصميم** □ النهج من أعلى إلى أسفل: تقسيم النظام إلى مجموعة من

أنظمة فرعية أصغر

□ النهج الموجه نحو الكائنات: تحديد مجموعة من الكائنات وتحديد تفاعلاتها. □
مخططات UML هي أداة تصميم لتوضيح

التفاعلات بين

□ الفصول الدراسية

□ الفئات والكيانات الخارجية

Software Life Cycle Activities

□ توليد الكود (التنفيذ).

يجب ترجمة التصميم إلى نموذج قابل للقراءة آليًا.

□ تُنجز خطوة توليد الكود هذه المهمة. إذا تم التصميم بدقة، □ يُمكن إنجاز توليد الكود

ميكانيكيًا.

Software Life Cycle Activities

□ الاختبار.

بمجرد إنشاء الكود، يتم اختبار البرنامج
تبدأ عملية الاختبار

□ يركز على الأجزاء الداخلية المنطقية للبرنامج، والتأكد من اختبار جميع البيانات، وعلى
الأجزاء الخارجية الوظيفية؛ أي إجراء الاختبارات للكشف عن الأخطاء والتأكد من أن
المدخلات المحددة ستنتج نتائج فعلية تتفق مع النتائج المطلوبة.

Software Life Cycle Activities

□ الدعم (الصيانة).

من المؤكد أن البرمجيات سوف تخضع للتغيير بعد أن يتم تطويرها.
تم تسليمها للعميل

(استثناء محتمل هو البرامج المضمنة). يحدث التغيير بسبب مواجهة أخطاء، أو بسبب ضرورة
تكييف البرنامج لاستيعاب التغييرات في بيئته الخارجية (مثل تغيير مطلوب بسبب نظام
تشغيل جديد أو جهاز طرفي جديد)، أو بسبب طلب العميل تحسينات وظيفية أو تحسينات
في الأداء. يُعيد دعم/صيانة البرنامج تطبيق كل مرحلة من المراحل السابقة على برنامج موجود
بدلاً من برنامج جديد.

Generic software process models

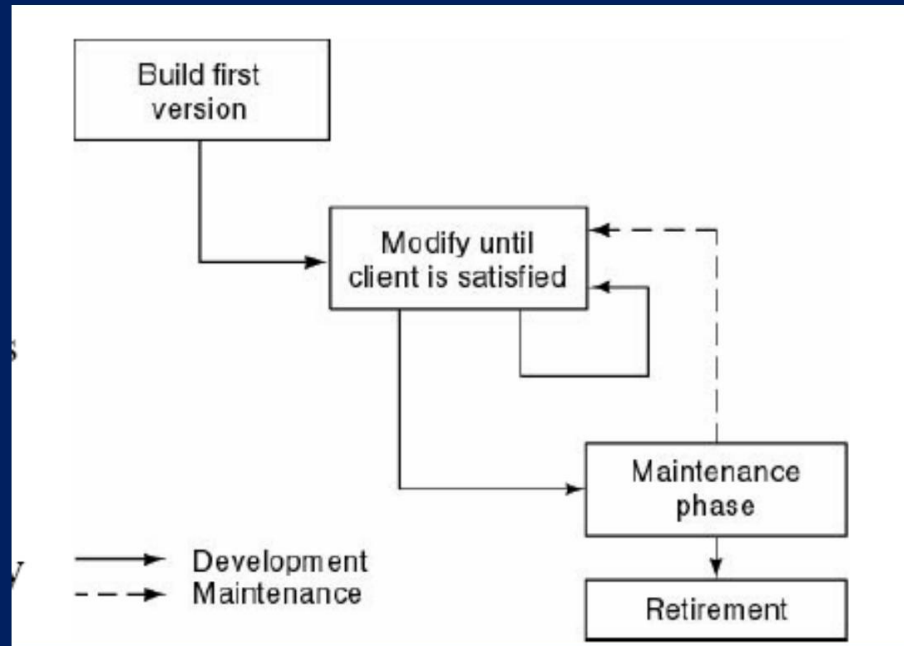
□ البناء والإصلاح □ الشلال □ التدريجي

□ القائم على المكونات Agile □

Build and Fix

في هذا النموذج البسيط لتطوير البرمجيات، يُبنى المنتج بمتطلبات بسيطة، وعادةً ما لا توجد مواصفات أو أي محاولة تصميم، وغالبًا ما يُهمل الاختبار. هذا تمثيل لما يحدث في العديد من مشاريع تطوير البرمجيات .

Build and Fix



Build and Fix

يبدأ هذا النموذج بفكرة عامة غير رسمية للمنتج، ويستمر في تطوير الشيفرة البرمجية حتى يصبح المنتج جاهزاً (أو حتى ينفد المال أو الوقت). يتم العمل بترتيب عشوائي .

Build and Fix

المزايا • لا توجد تكاليف إدارية

• علامات تقدم (البرمجة) مُبكرة. • خبرة محدودة، يُمكن لأي شخص استخدامها! • مُفيد لمشاريع "إثبات المفهوم" الصغيرة .

Build and Fix

العيوب 1. خطر عدم وجود رؤية/تحكم عدم وجود تخطيط للموارد عدم وجود مواعيد نهائية أخطاء يصعب اكتشافها/تصحيحها 2. مستحيل للمشاريع الكبيرة.

3. غالبًا ما يؤدي إلى إنتاج منتج ذي جودة منخفضة بشكل عام. 4. يمكن أن تكون الصيانة صعبة للغاية بدون مواصفات ووثيقة تصميم.

Waterfall Model

□ نموذج الشلال هو نموذج دورة الحياة الكلاسيكي - وهو معروف على نطاق واسع ومفهوم ويُستخدم بشكل شائع.

□ في بعض النواحي، يعتبر الشلال هو النهج "المنطقي".

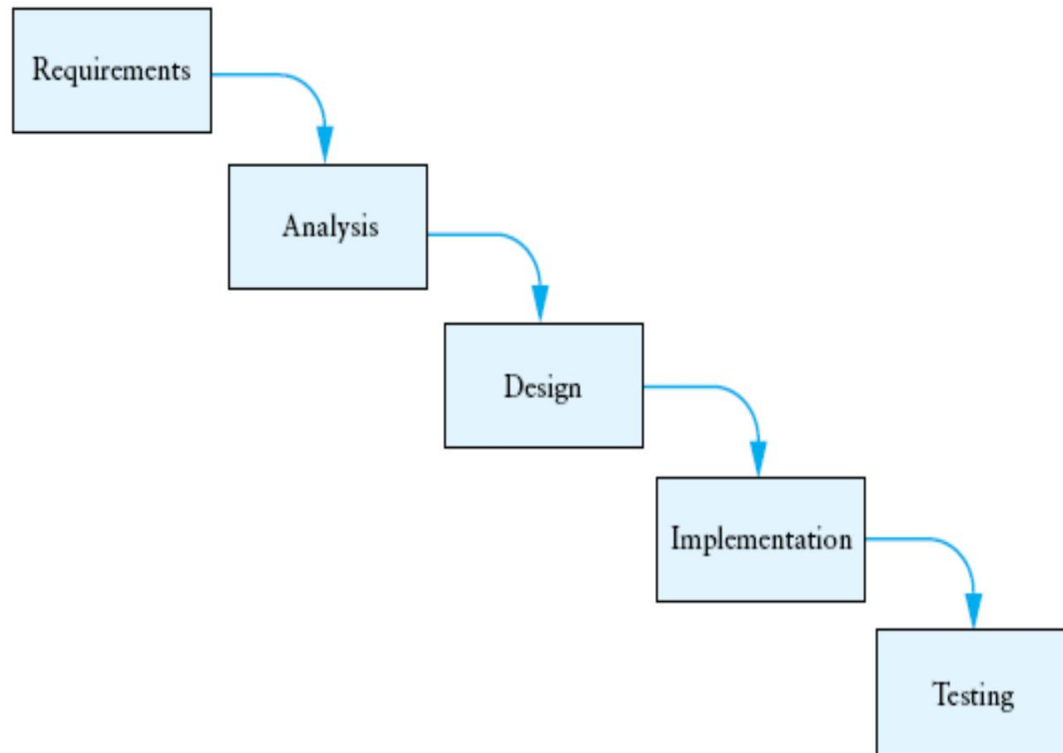
□ موجه نحو المستندات

□ يشكل مخرج إحدى المراحل المدخلات للمرحلة التالية (بطانة).

Waterfall Model

FIGURE 1.1

The Waterfall Software Life Cycle Model



Waterfall Model

الميزة □ سهولة الفهم، وسهولة الاستخدام □ توفير هيكل للموظفين عديمي الخبرة
□ يتم فهم المعالم جيدًا

□ جيد للتحكم في الإدارة (الخط، الموظفين،
مسار)

□ يعمل بشكل جيد عندما تكون الجودة أكثر أهمية من التكلفة أو الجدول الزمني

Waterfall Model

العيوب □ يجب معرفة جميع المتطلبات مقدمًا □ تعتبر المنتجات النهائية التي تم إنشاؤها لكل مرحلة مجمدة -مما يعوق المرونة □ يمكن أن يعطي انطباعًا خاطئًا بالتقدم □ لا يعكس طبيعة حل المشكلات في تطوير البرمجيات -تكرار المراحل □ فرصة ضئيلة للتعديل لمعاينة النظام (حتى فوات الأوان)

Incremental Model

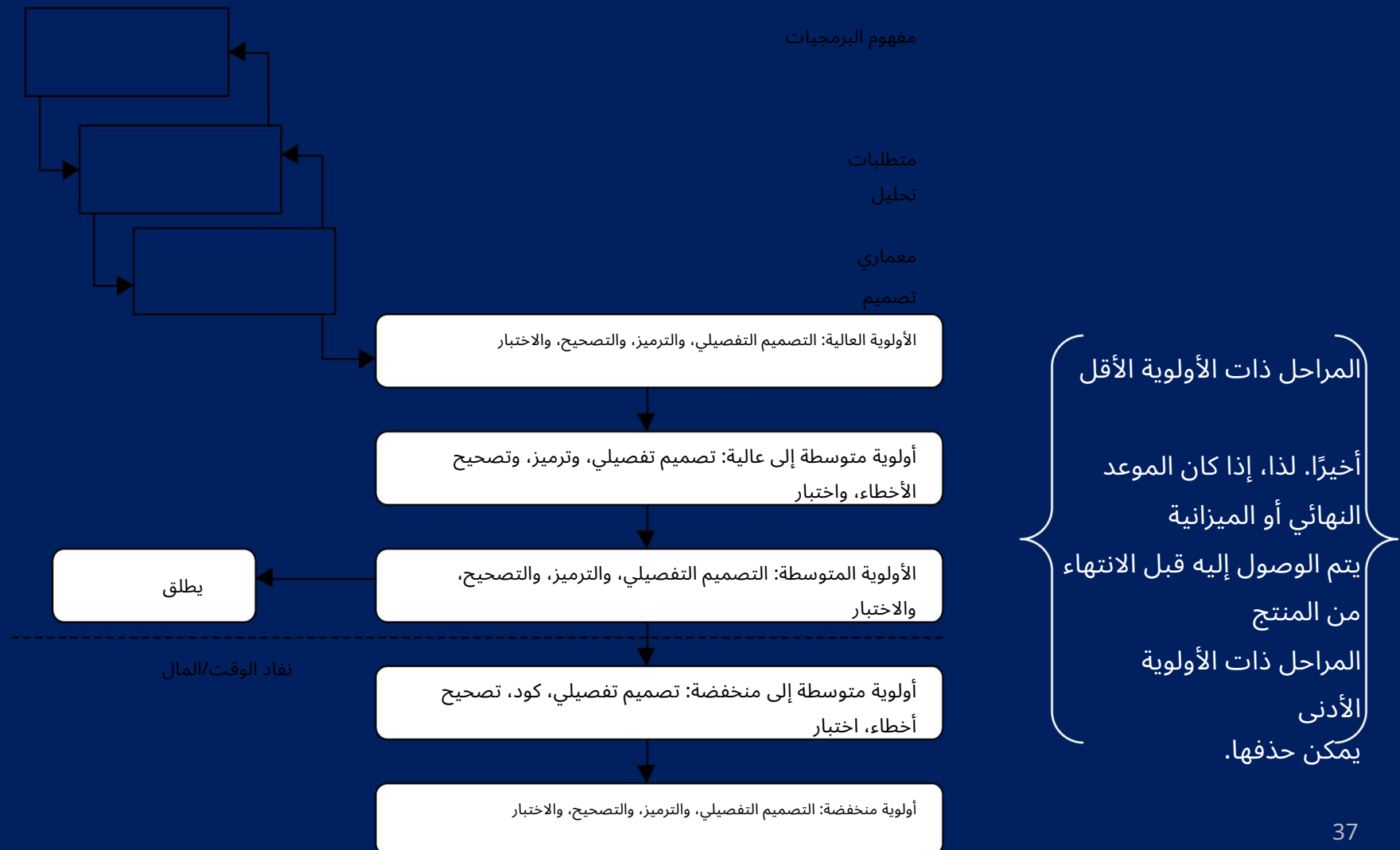
□ إنشاء تنفيذ جزئي لمجموع

النظام □ ثم قم بإضافة الوظائف المتزايدة ببطء □ يعطي النموذج التدريجي الأولوية لمتطلبات النظام ثم ينفذها في مجموعات.

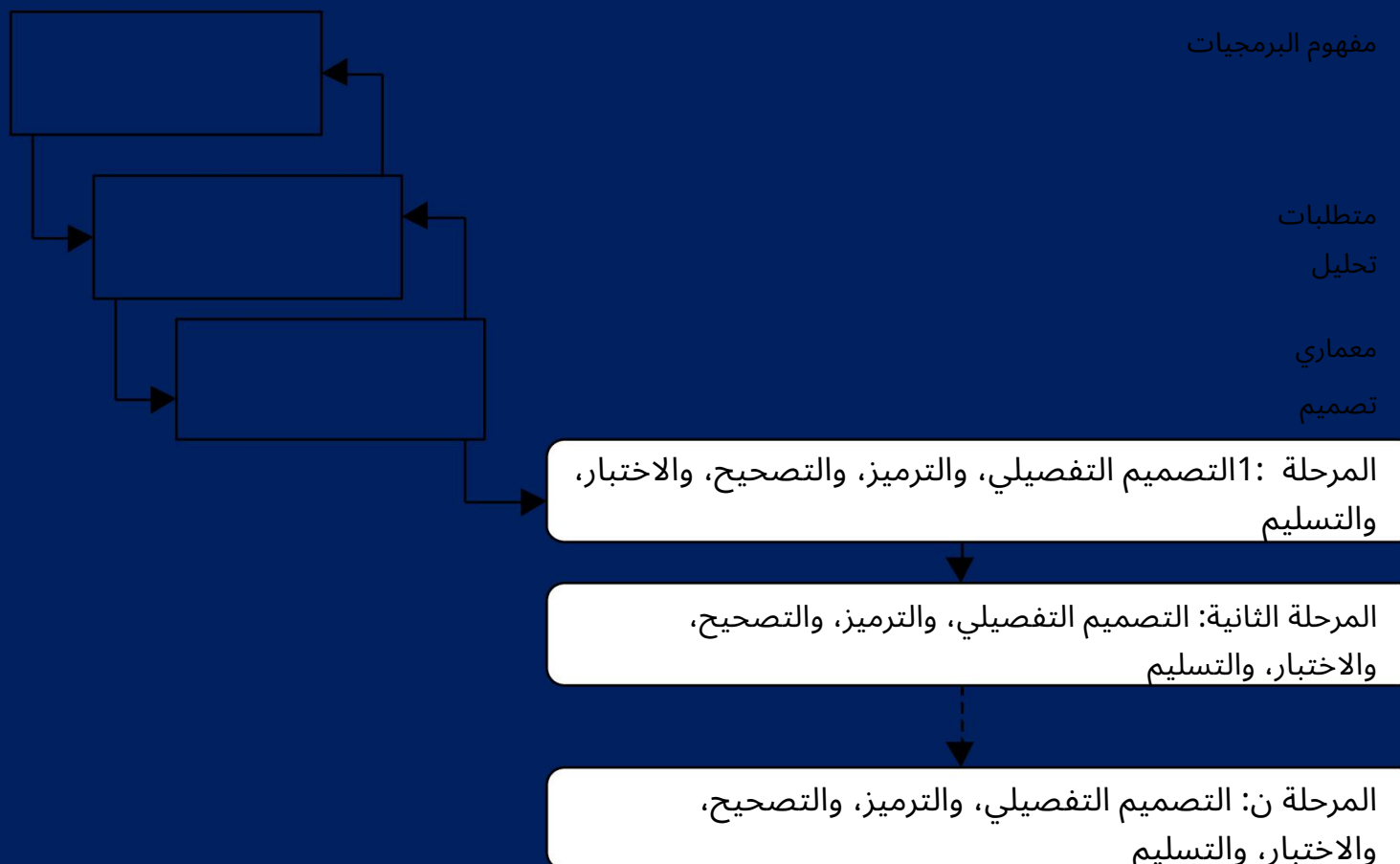
يضيف كل إصدار لاحق من النظام وظيفة إلى الإصدار السابق، حتى يتم تنفيذ جميع الوظائف المصممة. يختلف عن النمذجة الأولية في أن المتطلبات

معروفة تمامًا في البداية

Incremental Model



Incremental Model



Incremental Model

المزايا

- قم بتطوير الوظائف عالية الخطورة أو الرئيسية أولاً
- يقدم كل إصدار منتجًا تشغيليًا □ يمكن للعميل الاستجابة لكل إصدار
- يقلل من تكلفة التسليم الأولية □ يكون تسليم المنتج الأولي أسرع
- يحصل العملاء على وظائف مهمة في وقت مبكر
- يتم تقليل مخاطر تغيير المتطلبات

Incremental Model

العيوب

□ يتطلب تخطيطًا وتصميمًا جيدًا

□ يتطلب تعريفًا مبكرًا لنظام كامل ووظيفي بالكامل للسماح بتحديد الزيادات

□ كل مرحلة من مراحل التكرار صارمة ولا تتداخل مع بعضها البعض. □
التكلفة الإجمالية للنظام الكامل ليست أقل

Component-based Model

ربت عي النهج القائم على المكونات نهجًا لتطوير البرامج يعتمد على إعادة استخدام البرامج.

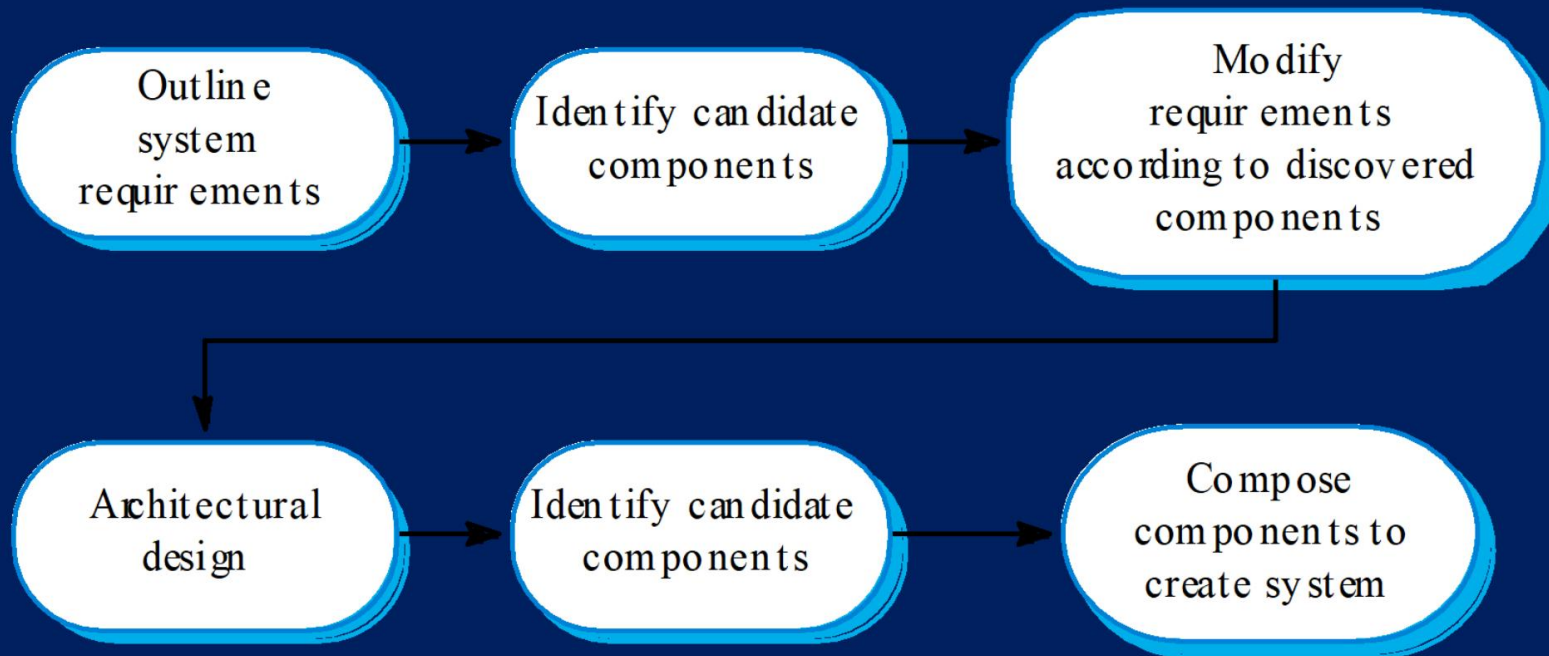
يعتمد على إعادة الاستخدام المنهجي حيث يتم دمج الأنظمة من أنظمة المكونات الموجودة. أصبح هذا النهج مستخدمًا بشكل متزايد مع ظهور معايير المكونات.

Component-based Model

□ يتضمن النموذج القائم على المكونات العديد من الخصائص التكرارية للنموذج
الحلزوني. □ الفرق الرئيسي هو أن التركيز في هذا النموذج ينصب على تكوين الحلول
من مكونات أو فئات برمجية معبأة مسبقًا

Component-based Model

عملية قاعدة المكونات



Component-based Model

المزايا: إدارة التعقيد □ تقليل
وقت التطوير □ زيادة الإنتاجية □
تحسين الجودة

Component-based Model

❑ العيوب: ❑ تطوير المكونات ❑ نقص
المكونات ❑ تكاليف صيانة المكونات ❑
المتطلبات غير المُلباة

Agile Manifesto

"أولويتنا القصوى هي إرضاء
"العميل من خلال التسليم المبكر والمستمر للبرامج القيمة"

An Agile Process

تعتمد المناهج الرشيقية على بعض المبادئ المشتركة، بعضها

□ يُعدّ عمل البرمجيات مقياسًا رئيسيًا لتقدم المشروع. □ لذلك، لتحقيق التقدم في المشروع، يجب تطوير البرمجيات وتسليمها بسرعة وبزيادات صغيرة.

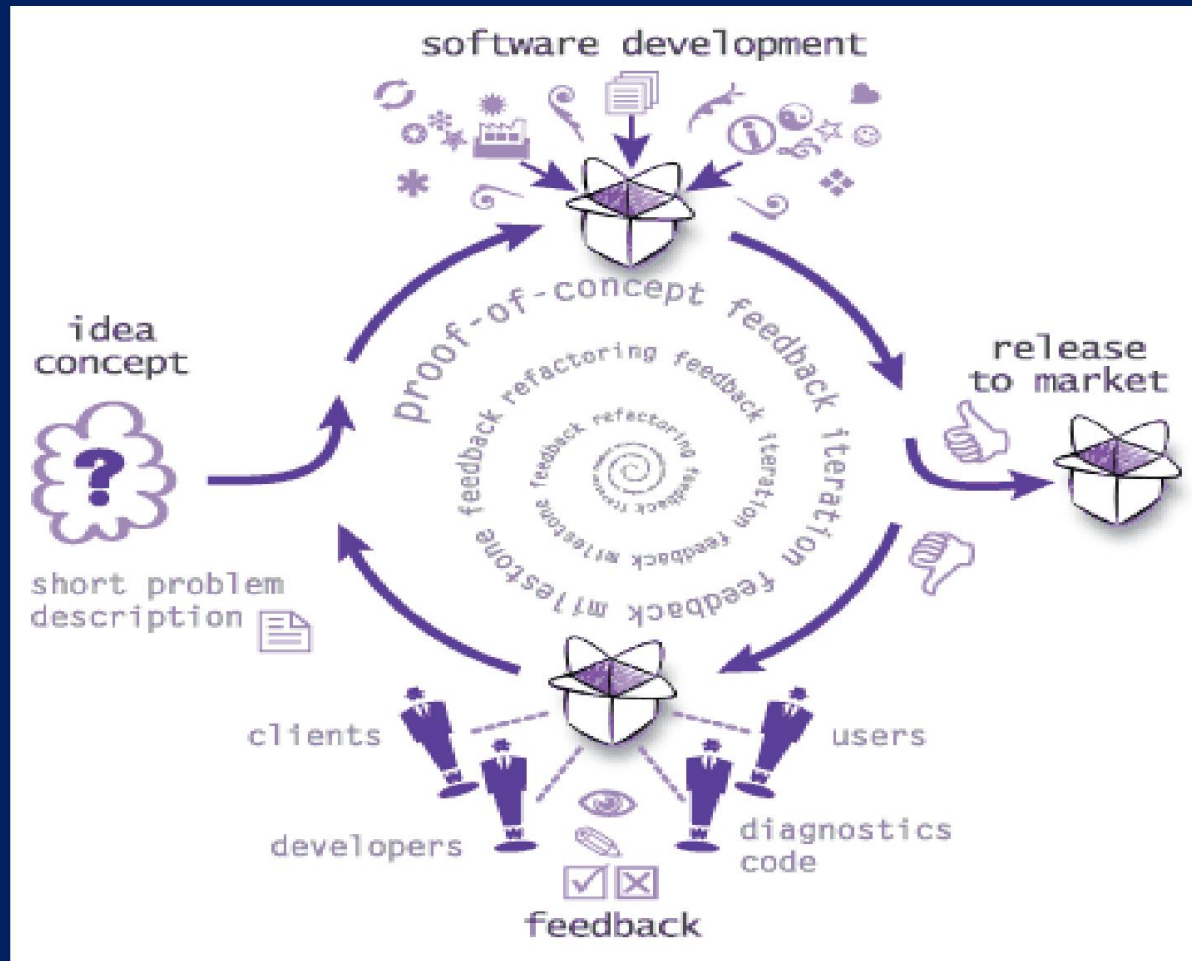
□ حتى التغييرات المتأخرة في المتطلبات يجب أن تؤخذ في الاعتبار

An Agile Process

□ يفضل التواصل وجهاً لوجه
التوثيق.

□ يُعَدُّ التقييم المستمر وإشراك العميل أمرًا ضروريًا لتطوير برامج عالية الجودة. □ يُعَدُّ التصميم البسيط
الذي يتطور ويتحسن مع مرور الوقت نهجًا أفضل من القيام بتصميم مُعَقَّد مُسَبِّقًا للتعامل مع جميع
السيناريوهات المُحتملة. □ تُحَدَّد مواعيد التسليم فرق مُؤَهَّلة من الأفراد الموهوبين.

An Agile Process



An Agile Process

بعض الأساليب الرشيقية

□ تطوير البرمجيات التكيفي □ (ASD) التطوير القائم على
الميزات □ Crystal Clear □ (FDD) طريقة تطوير
البرمجيات الديناميكية

□ (DSDM) تطوير التطبيقات السريع (RAD)

□ سكروم

□ البرمجة المتطرفة (XP)

Extreme Programming - XP

□ بالنسبة للفرق الصغيرة والمتوسطة الحجم التي تعمل على تطوير برامج ذات متطلبات غامضة أو سريعة التغير □ يعد الترميز هو النشاط الرئيسي طوال المشروع

مشروع برمجي

□ يتم التواصل بين زملاء الفريق باستخدام الكود

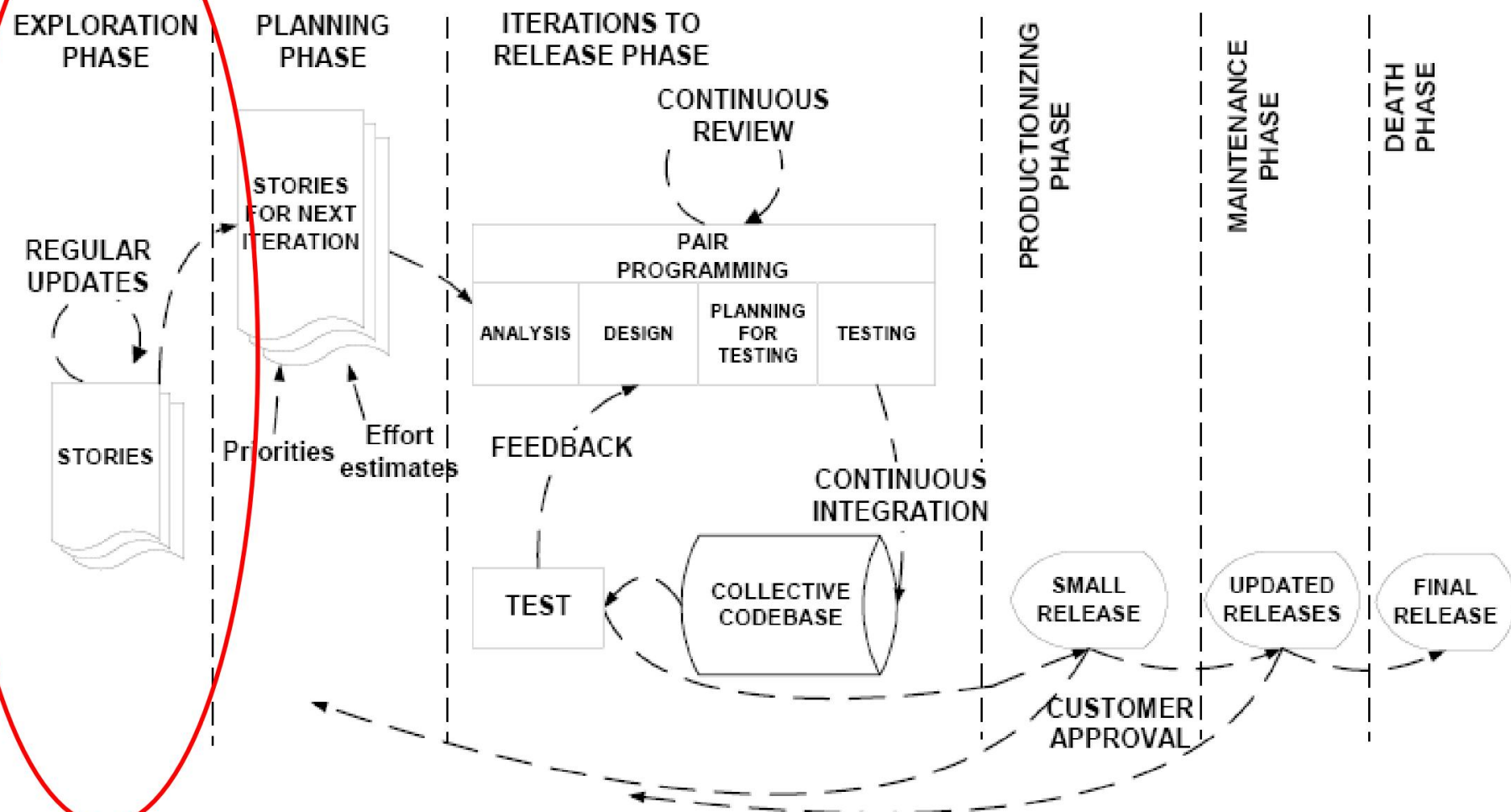
□ دورة حياة وسلوك الكائنات المعقدة المحددة في حالات الاختبار - مرة أخرى في الكود

Extreme Programming (XP)

يبدأ مشروع البرمجة المتطرف بقصص المستخدم التي تكون عبارة عن أوصاف قصيرة (بضع جمل) للسيناريوهات التي يرغب العملاء والمستخدمون في أن يدعمها النظام.

□ قصة المستخدم هي وصف قصير لما تريد الشركة أو العميل أن يفعله البرنامج، يكتبه العميل بمصطلحات العميل دون استخدام قواعد تقنية

دورة حياة XP



Extreme Programming (XP)

مثال على قصة المستخدم

"أخبرني القصة واكتب اسم القصة وفقرة أو فقرتين."

إنفاذ الترخيص

عند تشغيل JeraWorks لأول مرة، يعرض مربع حوار الترخيص، ولن يتم المتابعة حتى يقوم المستخدم بإدخال أي مما يلي:

- شهادة ترخيص سارية المفعول غير محدودة المدة (مدفوعة الأجر) أو
- ترخيص صالح لفترة محدودة (تجريبي) ولم ينته بعد.

يتم تخزين الترخيص الصالح حتى لا يضطر المستخدم إلى إعادة إدخاله في عمليات التشغيل اللاحقة.

يتم عرض معلومات الترخيص على شاشة البداية.

عند انتهاء صلاحية ترخيص العرض التوضيحي، يظهر مربع حوار الترخيص مرة أخرى في المرة التالية التي يتم فيها تشغيل JeraWorks.

Extreme Programming (XP)

اثنا عشر تمرينًا للحصول على الخبرة

□ لعبة التخطيط □ الإصدارات الصغيرة

□ البرمجة الزوجية □ الملكية الجماعية □ مستمرة

□ الاستعارة □ التصميم البسيط □

التطوير الموجه بالاختبار □ إعادة
الهيكل

التكامل 40 □ ساعة في الأسبوع

□ العمل في الموقع

□ معايير الترميز

Extreme Programming (XP)

لعبة التخطيط □ حدد بسرعة نطاق الإصدار التالي، من خلال الجمع بين

أولويات الأعمال والتقديرات الفنية.

□ قرارات العمل (العميل) □ النطاق: ما هي "القصص" التي ينبغي تطويرها □ أولوية القصص (الميزات) □

تواريخ الإصدار

□ القرارات الفنية (المطورون) □ تقديرات الوقت للميزات/القصص □ توضيح عواقب

القرارات التجارية □ تنظيم الفريق والعملية □ الجدولة

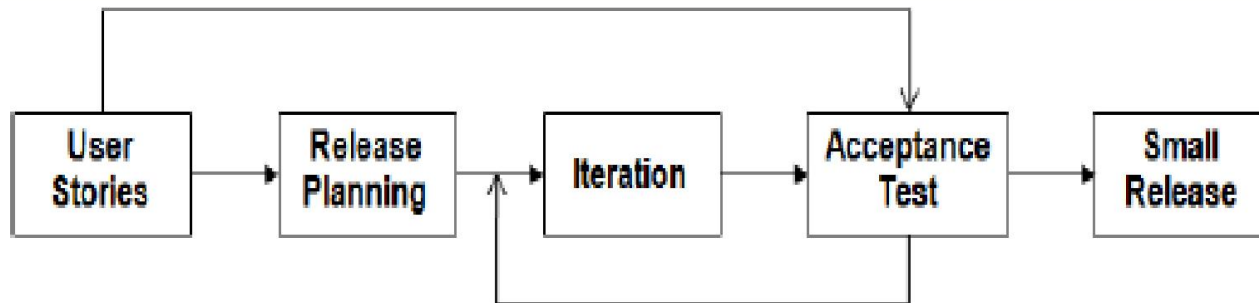
Extreme Programming (XP)

إصدار صغير

□ يتم تشجيع الإصدارات المتكررة والصغيرة، ويتم استخدام التكرارات للإصدار الواحد.

□ ابدأ بأصغر مجموعة من الميزات المفيدة

□ الإصدار المبكر وفي كثير من الأحيان، مع إضافة بعض الميزات في كل مرة



Extreme Programming (XP)

الاستعارة □ يمتلك المبرمجون قصة مشتركة بسيطة تشرح النظام □ استخدم
الاستعارات لوصف كيفية عمل النظام

عمل.

□ يتم تحديد شكل النظام من خلال استعارة أو مجموعة من الاستعارات المشتركة
بين العميل والمبرمجين. □ مجموعة من الاستعارات البسيطة تحدد البنية الداخلية
والخارجية للنظام. □ فهي تساعد الجميع على فهم العناصر الأساسية

Extreme Programming (XP)

مثال على استعارة XP- نظام الرواتب يشبه خط التجميع:

كان أحد مشاريع XP الكبيرة عبارة عن نظام رواتب لشركة كرايسلر (شركة سيارات). كان الاستعارة لهذا المشروع هي أن نظام الرواتب كان أشبه بخط تجميع حيث تم تحويل أجزاء الساعة إلى أجزاء الدولار، وتم تجميع جميع الأجزاء وتم إنتاج شيك راتب.

Extreme Programming (XP)

تصميم بسيط □ قم بأبسط شيء ممكن

عمل

□ يجتاز جميع الاختبارات

□ لا يوجد كود مكرر □ أقل عدد ممكن من الفئات والطرق

Extreme Programming (XP)

التطوير الموجه بالاختبار □ اختبر أولاً: قبل إضافة ميزة، اكتب اختبارًا لها! □ عندما يجتاز مجموعة الاختبار الكاملة 100%، يتم قبول الميزة □ تأتي الاختبارات بنوعين أساسيين... □ تعمل اختبارات الوحدة على أتمتة اختبار الوظيفة أثناء كتابتها من قبل المطورين □ يختبر كل اختبار وحدة عادةً فئة واحدة فقط، أو مجموعة صغيرة من الفئات!

□ يتم تحديد اختبارات القبول (أو الاختبارات الوظيفية) من قبل العميل

Extreme Programming (XP)

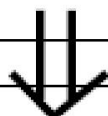
إعادة الهيكلة □ إعادة هيكلة البرمجيات دون تغيير سلوكها.

إعادة هيكلة النظام بشكل مستمر لتحسين الكود والقضاء على التكرار
□ تحسين جودة الكود بشكل مستمر

Extreme Programming (XP)

مثال على قاعة الطعام

```
void printFormatted(string text) {  
    System.out.println("Copyright (c) 1006, Siemens AG");  
    System.out.println("Author: Michael Stal");  
    printRest(text);  
}
```



```
void printFormatted(string text) {  
    printHeader();  
    printRest(text);  
}  
  
printHeader() {  
    System.out.println("Copyright (c) 1006, Siemens AG");  
    System.out.println("Author: Michael Stal");  
}
```

Extreme Programming (XP)

البرمجة الزوجية □ يعمل مبرمجان معًا على جهاز واحد □ يدخل السائق الكود، بينما يقوم المستكشف بمراجعته □ تبادل الأدوار بشكل دوري



نتائج البحث:



البرمجة الزوجية تزيد الإنتاجية

كود ذو جودة أعلى (15% عيوب أقل) في حوالي نصف الوقت (58%)

Extreme Programming (XP)

ملكية الكود الجماعية

لا يوجد شخص واحد "يملك" وحدة نمطية يمكن لأي مطور العمل على أي جزء من قاعدة التعليمات البرمجية في أي وقت

Extreme Programming (XP)

التكامل المستمر □ يتم دمج جميع التغييرات في قاعدة التعليمات البرمجية
يوميًا على الأقل □ يجب تشغيل الاختبارات بنسبة 100% قبل

وبعد التكامل

Extreme Programming (XP)

لا تعمل أكثر من 40 ساعة في الأسبوع كقاعدة عامة.

• لا تعمل أبدًا لساعات إضافية لمدة أسبوعين متتاليين.

• اعمل فقط لعدد الساعات التي يمكنك أن تكون منتجًا فيها وعدد الساعات التي يمكنك الاستمرار فيها.

Extreme Programming (XP)

العمل في الموقع

□ يمكن لفريق البرمجة الوصول إلى العميل في جميع الأوقات

Extreme Programming (XP)

معايير الترميز

• الجميع يبرمجون وفقًا لنفس المعايير

تحافظ معايير الترميز على اتساق الكود وسهولة قراءته وإعادة صياغته من قبل الفريق بأكمله.

Extreme Programming (XP)

□ لماذا المتطرفة؟

□ إذا كانت مراجعات الكود جيدة، فسنقوم بمراجعة الكود طوال الوقت (البرمجة الزوجية).

□ إذا كان الاختبار جيدًا، فسيقوم الجميع بالاختبار طوال الوقت (اختبار الوحدة)، حتى العملاء (الاختبار الوظيفي)

□ إذا كان التصميم جيدًا، فسنجعله جزءًا من العمل اليومي للجميع (إعادة الهيكلة)

□ إذا كانت البساطة جيدة، فسوف نترك النظام دائمًا بالتصميم الأبسط الذي يدعم الوظائف الحالية (أبسط شيء يمكن أن يعمل).

Extreme Programming (XP)

□ إذا كانت الهندسة المعمارية مهمة،

سيعمل الجميع على تحديد وصقل الهندسة المعمارية طوال الوقت (استعارة)

□ إذا كان اختبار التكامل مهمًا، فسنقوم بالتكامل والاختبار عدة مرات في اليوم (التكامل المستمر).

□ إذا كانت التكرارات القصيرة جيدة، فسنجعل التكرارات قصيرة جدًا، جدًا -ثوانٍ ودقائق وساعات، وليس أسابيع وأشهر وسنوات (لعبة التخطيط).

Extreme Programming (XP)

مزايا XP

التركيز على العملاء يزيد من فرصة أن البرنامج المنتج سوف يلبي بالفعل احتياجات المستخدمين . التركيز على الإصدارات الصغيرة التدريجية يقلل من المخاطر على مشروعك: من خلال إظهار أن نهجك يعمل و من خلال وضع الوظائف في أيدي المستخدمين، مما يمكنهم من تقديم ملاحظات في الوقت المناسب بشأن عملك. يساعد الاختبار والتكامل المستمر على زيادة جودة عملك . XP جذاب للمبرمجين الذين لا يرغبون عادةً في تبني عملية برمجية، مما يتيح لمنظمتك إدارة جهودها البرمجية بشكل أفضل.

Extreme Programming (XP)

عيوب XP

XP مُصمم لمشروع واحد، يُطوره ويحافظ عليه فريق واحد. XP مُعرّض للخطر بشكل خاص للمطورين "الفاسدين" الذين: لا يتعاونون جيدًا مع الآخرين الذين يعتقدون أنهم يعرفون كل شيء، و/أو غير راغبين في مشاركة أكوادهم "المتفوقة". لن يعمل XP في بيئة يُصرّ فيها العميل أو المدير على مواصفات أو تصميم كامل قبل بدء البرمجة. لن يعمل XP في بيئة يكون فيها المبرمجون منفصلين جغرافيًا. لم يُثبت أن XP يعمل مع أنظمة بها مشكلات في قابلية التوسع (يجب دمج التطبيقات الجديدة في الأنظمة الحالية).



How the customer explained it



How the Project Leader understood it



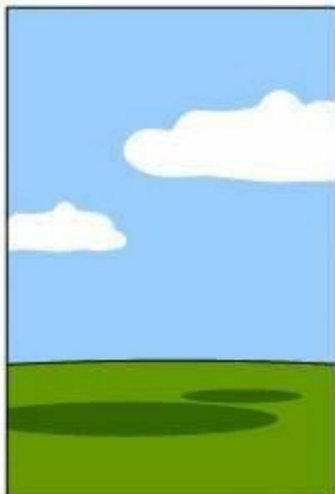
How the Analyst designed it



How the Programmer wrote it



How the Business Consultant described it



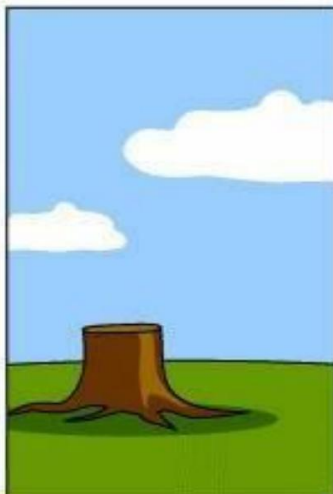
How the project was documented



What operations installed



How the customer was billed



How it was supported



What the customer really needed